Technical Report: Migrating a File Transfer Application from TCP to RPC

Do Hung Anh - 23BI14015

December 7, 2025

Abstract

This report documents the process of re-engineering a basic file transfer application from a low-level TCP socket implementation to a high-level Remote Procedure Call (RPC) framework. It outlines the architectural changes, the step-by-step transformation process, the challenges faced during development, and the solutions implemented. The report concludes with suggestions for future enhancements to improve performance, error handling, and security.

1 Introduction

Direct TCP socket programming provides maximum control over network communication but requires developers to manually handle data serialization, message framing, and application-level protocols. This is often complex and error-prone.

Remote Procedure Call (RPC) offers a higher level of abstraction. It allows a client to execute a procedure in a different address space (typically on another machine) as if it were a local function call. The RPC framework automatically handles the underlying network communication, data marshalling (serialization), and unmarshalling. This migration project aimed to leverage the simplicity and robustness of RPC for a file transfer application.

2 The Transformation Process

The migration from TCP to RPC involved a fundamental shift from managing data streams to defining a clear remote API. The following steps were taken:

1. **Define the Service Interface:** The first step was to create an RPC interface definition file (`filetransfer.x`). This file formally specifies the remote procedures (`UPLOAD_FILE`, `DOWNLOAD_FILE`) and the data structures they use. This replaces manual parsing of commands from a TCP stream.
2. **Generate Stub and Skeleton Code:** Using the `rpcgen` tool, the `.x` file was compiled to automatically generate:
   - Client-side "stubs" that translate local function calls into network messages. - Server-side "skeletons" that receive network messages and call the actual service implementation.
   - Data serialization/deserialization routines (XDR code).
3. **Implement Server-Side Logic:** The generated server skeleton provides empty functions (e.g., `download_file_1_svc`). The core logic for reading files from the server's filesystem was implemented within these functions.
4. **Implement Client-Side Logic:** The client application was rewritten to call the high-level stub functions (e.g., `download_file_1()`) instead of using `socket()`, `connect()`, `send()`, and `recv()`.
5. **Compile and Link:** The build process was updated to compile all generated files and link against the necessary RPC library (`libtirpc`).

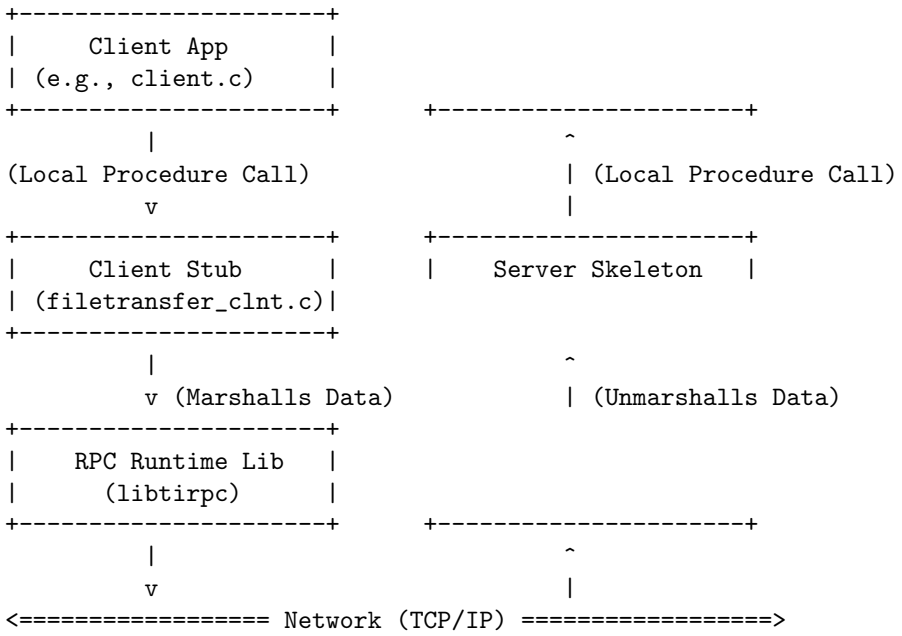

Figure 1: RPC File Transfer Architecture

3 RPC Architecture Diagram

The RPC model introduces a clear separation between the application logic and the network communication code. The diagram below illustrates this layered architecture.

4 Challenges Faced and Solutions

The migration process presented several technical challenges.

4.1 RPC Interface Definition and Code Generation

- **Challenge:** The `rpcgen` tool did not initially generate the expected C `struct` for the download arguments. It produced a simple `typedef` to a character pointer, ignoring the `offset` field needed for chunking.
- **Solution:** The interface definition in `filetransfer.x` was modified to use a more explicit C-style declaration. This forced `rpcgen` to correctly generate the desired struct, enabling the client to send both the filename and the file offset.

4.2 Build Environment and Library Linking

- **Challenge:** The initial compilation failed because the compiler could not find the required header file `rpc/rpc.h`.
- **Solution:** Investigation revealed that the modern RPC implementation, `libtirpc`, installs its headers in a non-standard directory (`/usr/include/tirpc`). The Makefile was modified to add the correct include path (`-I/usr/include/tirpc`) and to link against `libtirpc` (`-ltirpc`) instead of the obsolete network services library (`-lnsl`).

4.3 Server-Side State and Concurrency

- **Challenge:** The original server-side download logic used a `static FILE*` pointer. This is a critical flaw, as it is not re-entrant and would cause race conditions and incorrect behavior if multiple clients attempted to download files concurrently.

- **Solution:** The server was re-architected to be stateless for each download chunk request. The client now sends the file offset with each request. The server performs an `fopen()`, `fseek()`, `fread()`, and `fclose()` sequence for every chunk. While this increases I/O overhead, it guarantees that concurrent requests are handled correctly and independently.

5 Recommendations for Improvement

While the current RPC implementation is functional and robust, several areas could be improved.

- **Efficient State Management:** The stateless open/close-per-chunk model on the server is safe but inefficient. A better approach would be to implement a session model. The client would make an initial "open file" call and receive a unique session handle. Subsequent "read chunk" calls would use this handle, allowing the server to keep the file open and mapped to that specific client, drastically reducing I/O overhead. A final "close file" call would release the resources.
- **Enhanced Error Reporting:** The current implementation has minimal error reporting. If a download fails, the client only knows that an error occurred. The RPC response structures should be enhanced to include an error code and a descriptive error message. This would allow the server to communicate specific failures (e.g., "File not found," "Permission denied") to the client for a better user experience.
- **Security:** The current implementation is entirely insecure; any client can connect and request files. A robust implementation should include:
  - **Authentication:** Add procedures for user login, returning a session token that must be passed with all subsequent file operation requests.
  - **Authorization:** The server should check user permissions before allowing file read or write operations.
- **Data Buffering:** The use of a static buffer on the server for reading file chunks is simple and avoids memory leaks. However, for more advanced scenarios, using dynamically allocated memory (with `malloc`) and ensuring it is properly freed by the RPC framework (using `xdr_free`) would provide more flexibility.

6 Conclusion

Migrating the file transfer application from TCP to RPC successfully abstracted away the complexities of network programming, resulting in cleaner, more maintainable code. The process highlighted the importance of understanding the code generation tools, managing build dependencies, and designing stateless or carefully stateful services for concurrent access. While the resulting implementation is functional, the recommended improvements in state management, error handling, and security would be essential for deploying such a system in a production environment.